Przekazywanie obiektów jako parametrów funkcji

Lekcja 15 - Programowanie aplikacji

Klasa 4 technikum - technik programista - kwalifikacja INF.04

```
Pracownik p;
pobierzDane(p);
wyswietlDane(p);

Pracownik* w = new Pracownik();
w->wyswietlDane();
delete w;
```

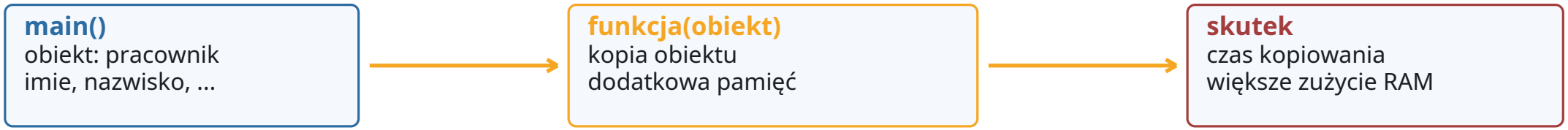
Cele lekcji

- wyjaśnisz, dlaczego przekazywanie dużych obiektów przez wartość może być kosztowne;
- rozróżnisz przekazywanie obiektu przez referencję i przez wskaźnik;
- zastosujesz const przy parametrach wejściowych, aby chronić obiekt przed zmianą;
- zapiszesz funkcje pobierające i wyświetlające dane obiektu;
- ocenisz, kiedy lepiej użyć referencji, a kiedy wskaźnika.

Powiązanie z INF.04.4
Programowanie obiektowe: stosowanie zasad programowania obiektowego, definiowanie klas i metod oraz bezpieczne operowanie obiektami.

Problem: kopiowanie całego obiektu

Obiekt może mieć rozbudowaną strukturę wewnętrzną. Gdy funkcja dostaje obiekt przez wartość, powstaje jego kopia.



- przy małych obiektach koszt bywa pomijalny;
- przy dużych obiektach kopiowanie może spowalniać program;
- zmiany wykonane na kopii nie zmienią oryginalnego obiektu.

Rozwiązanie: referencje albo wskaźniki

Dla dużych obiektów parametry funkcji definiujemy zwykle jako referencje albo wskaźniki. Wtedy funkcja pracuje na istniejącym obiekcie, a nie na jego kopii.

Referencja
Przekazuje dostęp do istniejącego obiektu. Składnia w funkcji wygląda jak zwykły obiekt: p.imie.

Wskaźnik
Przechowuje adres obiektu. Do pól obiektu odwołujemy się operatorem ->.

const
Oznacza, że funkcja nie powinna zmieniać przekazanego obiektu.

```
void pokaz(const Pracownik& p);
Pracownik& pobierzDane(Pracownik& p);
```

```
void pokaz(const Pracownik* p);
Pracownik* pobierzDane(Pracownik* p);
```

Parametry wejściowe i wyjściowe

Parametr wejściowy
Służy do odczytu danych przez funkcję. Najbezpieczniej przekazać go jako const referencję lub const wskaźnik.

Parametr wyjściowy
Służy do przekazania danych z funkcji do otoczenia. Funkcja może zmienić obiekt przez referencję albo wskaźnik.

```
void wyswietlDane(const Pracownik& p)
{
    cout << p.imie;
}
```

```
Pracownik& pobierzDane(Pracownik& p)
{
    cin >> p.imie;
    return p;
```

Przykład 11.11: przekazanie przez referencję

```
#include <iostream>
using namespace std;

class Pracownik {
public:
    string imie;
    string nazwisko;
};

Pracownik& pobierzDane(Pracownik& p) {
    cout << "Imię = "; cin >> p.imie;
    cout << "Nazwisko = "; cin >> p.nazwisko;
    return p;
}

void wyswietlDane(const Pracownik& p) {
    cout << "Imię: " << p.imie << endl;
    cout << "Nazwisko: " << p.nazwisko << endl;
}

int main() {
    Pracownik pracownik;
    pobierzDane(pracownik);
    wyswietlDane(pracownik);
}
```

Co dzieje się w przykładzie z referencją?

Referencja jest drugim sposobem dostępu do tego samego obiektu. Funkcja nie tworzy kopii obiektu pracownik.




```
p.imie = "Jan";      // zmiana oryginału
cout << p.nazwisko;  // odczyt pola
```

```
void wyswietlDane(const Pracownik& p)
// brak kopii + ochrona przed zmianą
```

Rola const

const informuje kompilator i programistę, że funkcja ma tylko odczytywać przekazany obiekt. To ważne przy parametrach wejściowych.

```
void wyswietlDane(const Pracownik& p) {
    cout << p.imie << " " << p.nazwisko;
    // p.imie = "Jan";   // błąd: obiekt jest tylko do odczytu
}

void pokaz(const Pracownik* p) {
    if (p != nullptr) cout << p->imie;
}
```

Wniosek
Jeżeli funkcja ma tylko wyświetlać lub sprawdzać dane, użyj const. Dzięki temu nie zmieni przypadkowo pól obiektu.

Skrót
RAM - pamięć operacyjna, w której podczas działania programu przechowywane są m.in. obiekty i ich kopie.

Przykład 11.12: przekazanie przez wskaźnik

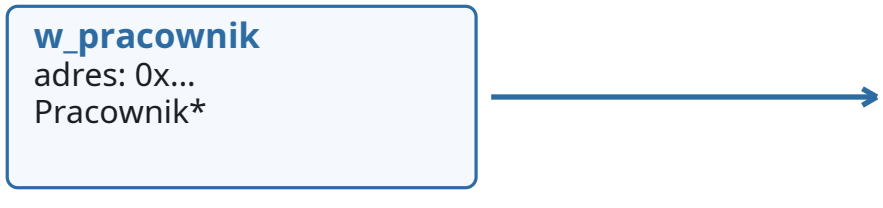
```
Pracownik* pobierzDane(Pracownik* w_p);
void wyswietlDane(const Pracownik* w_p);

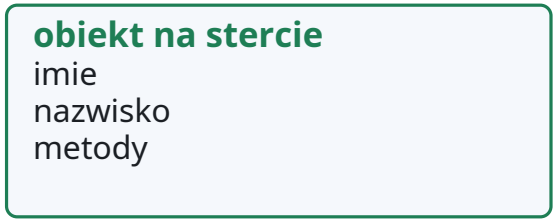
int main() {
    Pracownik* w_pracownik = new Pracownik();
    pobierzDane(w_pracownik);
    wyswietlDane(w_pracownik);
    delete w_pracownik;
    return 0;
}

Pracownik* pobierzDane(Pracownik* w_p) {
    cout << "Imię = "; cin >> w_p->imie;
    cout << "Nazwisko = "; cin >> w_p->nazwisko;
    return w_p;
}
```

Wskaźnik przechowuje adres obiektu

Wskaźnik nie jest obiektem, ale zmienną przechowującą adres obiektu. Do pól wskazywanego obiektu przechodzimy operatorem strzałkowym ->.






```
w_pracownik->imie = "Jan";
w_pracownik->nazwisko = "Kowalski";
w_pracownik->wyswietlDane();
```

Uwaga praktyczna
Obiekt utworzony przez new trzeba zwolnić przez delete. W nowoczesnym C++ częściej stosuje się obiekty automatyczne albo inteligentne wskaźniki, ale tutaj ćwiczymy mechanizm z podręcznika.

Referencja czy wskaźnik?

Sytuacja	Lepszy wybór	Dlaczego
Funkcja ma dostać istniejący obiekt	**referencja**	prostsza składnia, brak nullptr
Funkcja tylko odczytuje obiekt	**const referencja**	brak kopii i ochrona przed zmianą
Obiekt może nie istnieć	**wskaźnik**	można sprawdzić wartość nullptr
Obiekt utworzony dynamicznie	**wskaźnik**	przechowuje adres obiektu na stercie

Najważniejsza zasada: nie kopiuj dużego obiektu bez potrzeby.

Typowe błędy

- Przekazywanie dużego obiektu przez wartość, gdy funkcja tylko go odczytuje.
- Brak const przy parametrze wejściowym, co pozwala na przypadkową zmianę danych.
- Użycie wskaźnika bez sprawdzenia, czy nie ma wartości nullptr.
- Utworzenie obiektu przez new i brak delete po zakończeniu pracy.
- Mylenie operatorów: kropka . dla obiektu/referencji, strzałka -> dla wskaźnika.

```
Pracownik p;      p.imie = "Jan";
Pracownik* w=&p;   w->imie = "Jan";
```

Podsumowanie

- Obiekt przekazany przez wartość jest kopiowany.
- Referencja i wskaźnik pozwalają pracować na istniejącym obiekcie.
- Parametry wejściowe dużych obiektów warto zapisywać jako const referencje lub const wskaźniki.
- Parametry wyjściowe mogą być referencjami albo wskaźnikami.
- Przy wskaźnikach pamiętaj o operatorze -> oraz o zwalnianiu pamięci przy new/delete.

Pytanie kontrolne
Jak zapiszesz funkcję, która tylko wyświetla dane obiektu Pracownik i nie może ich zmienić?